Git in five minutes

NSDF-Data student guide 1 — for every computer

Version 5 · 12 November 2026

Tip

This is guide 1: start here. It explains, in plain words, how the whole group shares one project without getting in each other's way. You do not need to know anything about Git, and you do not need to type anything yet: just read. It takes about ten minutes. **You can skip it** if you already know what a commit, a push, a branch, a remote and a pull request are, and why the shared branches are protected, and if you already have a GitHub account and the lead's invitation. You can come back to it any time a Git word or message puzzles you.

Why this guide

Many people work on this project at the same time: the project lead, other students, the people who check the work. Everyone needs the same analysis code, and everyone makes changes. Without a system, two people change the same file and one loses their work, or a half-finished change breaks the website for everybody.

The system (a computer program) is called **Git**. Everything below is the few parts of it you need, and the reason for each rule.

You may know Google Docs, where several people type in one file at the same time. That suits an essay, but not an analysis. Everyone sees your half-finished typing as you type it, nothing says which version produced which result, and nobody checks a change before it counts. Here a change must be finished and read by a second person before it is added, and every earlier version must stay recoverable so that a result can be reproduced. That is what Git gives us.

The idea

Git keeps every saved version of your project, like an endless "Save As" where each save has a note saying what changed. Nothing you have saved is lost, even if you change your mind later. **GitHub** is a website (github.com) that keeps a shared copy of the project online, so that everyone, and your lead, can see the same thing.

Two copies: your computer and the remote

This is the idea that makes the rest easy. There are always at least two copies of the project:

Copy	Where it is	What it is called
Your copy	The folder `NSDF-Data` on your own computer.	**local** (the copy "here")
The shared copy	On GitHub's computers, on the internet.	**the remote** (the copy "over there"); Git calls it `origin`

Git does not keep them in step by itself. You tell it to send things from one to the other:

- **clone**: the first download of the project from the remote to your computer. The setup guide does it once.
- **push**: send the versions you saved on your computer up to the remote.
- **pull** (or **fetch**): bring newer versions from the remote down to your computer.

Saving a version on your computer (a **commit**) does not change the remote at all. Nobody else sees it until you push. That is the safety net: you can save, try, and change your mind on your own computer, and the shared copy stays untouched.

The words

Word	What it means here
repository	The project folder (`NSDF-Data`) together with its whole saved history.
commit	One saved version, with a short message saying what changed.
branch	A separate line of saved versions: a copy of the whole project where you can work without changing the original.
local and **remote**	Your computer's copy, and the copy on GitHub (see above).
push	Send your commits from your computer to the remote.
pull request	A button on github.com that means "I'm done, please have a look." Your lead reads your changes and, if they are ready, adds them to the project. Guide 7 shows it step by step.
merge	Adding the changes of one branch into another.
terminal	A window where you type commands. The setup guide installs what you need and shows how to open it.

Branches, and the three that matter

A **branch** is how Git lets many people work at once. Each person works on their own branch, so changes stay separate until they are deliberately put together.

This project has three kinds of branch you will meet:

Branch	What it is	Who changes it
`main`	The released version of the project: tested and numbered (such as 0.12.0).	Only the project lead, when a release is made.
`develop`	The shared working version, where everyone's finished work is collected. **The project website and the guides you are reading are served from it.**	Only through pull requests.
`student-yourname`	**Your own branch.** Yours alone; the setup guide (Step 8) creates it.	You.

`main` and `develop` are protected, and why

Protected means GitHub refuses to let anyone change `main` or `develop` directly. Nobody can push to them, **not even the project lead**. The only way a change gets in is a pull request that someone else has read and approved. GitHub also refuses to delete them or to rewrite their history.

The reasons:

- **They are what everyone sees.** The website and the guides come from `develop`. A change there appears online about two minutes later. A mistake there is public and affects everybody.
- **Everyone depends on them.** Each person builds their own work on top of `develop`. If it breaks, every one of you is stuck.
- **A second pair of eyes catches mistakes.** A wrong number, a missing file or a data file that should never be shared is easy to miss in your own work. A pull request is where someone else checks it.
- **Nothing is lost.** Because history cannot be rewritten, there is always a record of who changed what, and any change can be undone.

This is also a safety net for you. If you push to the wrong branch by accident, GitHub says no and nothing breaks.

Careful

GitHub protects the **remote** copies only. On your own computer nothing stops you from committing while you are on `develop`. The commit would be refused when you tried to push it, and then you would have to ask your lead to help. So always check which branch you are on before you commit (see the commands below).

How your work gets in: your branch, pushed to the remote

You never change `develop` yourself. You work on your own branch, send it to the remote, and ask for it to be added:

1. **Work on your branch, on your computer.** Edit a notebook or write a note. Save versions

(commits) as you go. Only your own copy changes.
2. **Push your branch.** This sends your commits to GitHub, where the remote now has a copy of your branch, named `origin/student-yourname`. Your lead and the others can see it. Nothing in `develop` has changed yet.
3. **Open a pull request** on github.com: "please add my branch to `develop`". Your lead reads the changes there, may ask you to fix something (you commit and push again, and the pull request updates itself), and when it is ready, merges it.
4. **Your work is now in** `develop`, which means in the shared project (and the website, if it was a note). Your branch stays, and you keep using it for your next piece of work.
5. **Bring the newest** `develop` **into your branch from time to time**, so you start from everyone's latest work (the section "Getting the newest project changes" below).

Your branch lives in both places: the copy on your computer, and the copy on the remote. A push makes the remote copy match your computer. Your branch cannot hurt anyone else's work or the public site, however many times you commit or push.

Your GitHub account, and what the invitation means

To push, you need two things: an account on GitHub, and permission from the project lead. Anyone can download (clone) the project, but only people the lead has invited can push to it.

1. Make a free account.

1. Go to github.com and choose **Sign up**.
2. Give an email address, a password and a **username**. Your username is public and is shown next to your work, so choose one you are happy for others to see (your name, or something close to it).
3. GitHub emails you a code to confirm the address. Type it in.
4. Keep your password private. Never type it into a terminal, and never send it to anyone, including the project lead.

2. Send your username to the project lead. Only the username (not your password).

3. The lead invites you. On the project's page on github.com, the lead opens **Settings**, then **Collaborators**, then **Add people**, and types your username. You become a **collaborator**.

4. Accept the invitation. GitHub sends an email to the address of your account, and shows a notice when you are signed in at github.com. Open it and choose **Accept invitation**. It expires after seven days. If it has, or you cannot find the email (look in your spam folder), tell the lead, who sends a new one.

What being a collaborator means. It gives you permission to **push** to the project's remote: to create your own branch there and send your commits to it. That is all. It does **not** let you change `main` or `develop`: those are protected for every collaborator, the lead included (see above). Your changes still reach them only through a pull request. You are being trusted to work on your own branch, not to publish. Collaborators can also see all of the project's pull requests and comment on them.

Why the invitation is needed at all. If anyone could push to the project, anyone could fill it with branches, or send files that should never be shared (such as data files). The invitation is the lead's way of saying "this person is part of the group", and it can be withdrawn later.

If you try to push before you have accepted, GitHub answers with `Permission denied` or `403`. Nothing is wrong with your computer: accept the invitation (or ask the lead), then push again. The setup guide (Step 8) takes you through the rest: telling Git your name and email, signing in to GitHub

from the terminal, and making your own branch.

You may hear about "Git Flow"

Git Flow is the name of this way of organising branches. In short: `main` holds released versions; `develop` collects finished work; everyone's own work happens on separate branches (for a student, `student-yourname`), which reach `develop` through pull requests; and the project lead makes short-lived `release/…` and `hotfix/…` branches to publish a numbered version or to fix something urgent. You only ever use your own branch and the pull request button. The rest is the maintainers' business.

The four commands

Once the setup guide (guide 2) has installed everything, you will use these all the time. Run them in a terminal, from inside the `NSDF-Data` folder (the setup guide shows how to get there). **Do not type them yet.**

```
git status
git add your-file
git commit -m "Say what you changed"
git push
```

Command	What it does
`git status`	Shows what has changed since your last commit, and which branch you are on. It only looks: it never changes anything, so use it as often as you like.
`git add your-file`	Chooses a file to include in your next commit. Use the real file name, as `git status` shows it.
`git commit -m "..."`	Saves a version (on your computer) of everything you added, with your message. A good message says what and why, in a few words.
`git push`	Sends your saved versions to the remote, on GitHub. The first time on a new branch, Git tells you the exact command to use: copy it.

Tip

Before every commit, type `git status` and read the list. It should show only your own files. If you see data files (a folder named `idx`, or `.bin` files) or other people's files, do not add them: ask your lead.

To see which branch you are on, type:

```
git branch --show-current
```

It should print `student-yourname`. If it prints `develop` or `main`, stop and ask your lead.

A normal session

1. Go to the `NSDF-Data` folder in a terminal.
2. Type `git branch --show-current` and check that it is your branch.
3. Do your work: a notebook, a note.
4. Type `git status`, then `git add` for each file of yours, then `git commit -m "..."`, then `git push`.
5. When a note or a piece of work is ready for review, open a pull request on github.com (guide 7, Step 3).

Getting the newest project changes

The project keeps changing: other people's work reaches `develop` all the time. To bring the newest shared changes into your branch, follow *Updating your setup* (guide 5, Step 2). It uses two lines, `git fetch origin` (look at what is new on the remote) and `git merge --no-edit origin/develop` (add it to your branch), and tells you what to do if Git reports a `CONFLICT`: stop, and send the message to your lead.

Prefer buttons to typing?

Everything above can also be done with buttons in VS Code. Guide 4 (*Git in VS Code*) shows how. It is optional: the terminal version stays the main one.

When something looks wrong

Careful

Stop, and do not try commands you found on the internet. Type `git status`, copy everything it prints, and send it to your lead together with the message that worried you. Work you have committed is very hard to lose, and nobody minds this question.

You see	What it means	What to do
`nothing to commit, working tree clean`	You have no unsaved changes.	Nothing: this is fine.
`Untracked files:`	Files Git has not been told about yet.	Add the ones that are yours. Leave data files alone.
`Your branch is ahead of 'origin/...' by 2 commits`	You committed but have not pushed.	`git push`
`Your branch is behind`	There are newer commits on the remote.	Ask your lead, or see guide 5, Step 2.
`CONFLICT`	Git cannot combine two changes by itself.	Stop. Do not edit anything. Send the message to your lead.
A text editor opens (a window full of ~ characters, or text starting `Merge branch`)	Git wants a message for a merge.	Press `Esc`, type `:wq`, press Enter.

You see	What it means	What to do
`fatal: not a git repository`	You are not inside the `NSDF-Data` folder.	Go to the `NSDF-Data` folder and try again.
`Please tell me who you are`, or a password or permission error when you push	Git does not know your name, or you are not signed in to GitHub.	Setup guide, Step 8. Never type your GitHub password into the terminal.
`Permission denied` or `403` when you push	You have not been added to the project yet, or you have not accepted the invitation.	Check your email (and spam) for the GitHub invitation and accept it; otherwise ask the lead. See “Your GitHub account”.
`protected branch`, or a push to `develop` or `main` is refused	You tried to change a protected branch directly.	This is the protection working. Do not look for a way around it. Check your branch with `git branch --show-current` and ask your lead.

Where to learn more

These are outside websites, which you can read at your own pace. None is required.

- **Software Carpentry, “Version Control with Git”**: a gentle course for scientists. https://swcarpentry.github.io/git-novice/
- **GitHub: About pull requests**: https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests
- **GitHub Skills**: short interactive courses in your browser. https://skills.github.com/
- **Pro Git**, the free book, for later. https://git-scm.com/book/en/v2
- **Dangit, Git!?!**: plain recipes for “I did something wrong”. https://dangitgit.com/

Session Info

Guide version 5, 12 November 2026. Written for every computer. Please tell the project lead where a step was unclear.

Where to get help

Anthony Villano, project lead: anthony.villano@ucdenver.edu

When you write, include the output of `git status` and the message that worried you.

Key links

- Repository: https://github.com/villano-lab/NSDF-Data
- Student guides: https://villano-lab.github.io/NSDF-Data/